Software Design Documentation: Risk Calculation System

1. System Overview

The Risk Calculation System is an object-oriented Python application designed to quantify the market risk of a portfolio containing stocks and European options. It computes Value at Risk (VaR) and Expected Shortfall (ES) using three distinct methodologies: Historical Simulation, Parametric (Delta-Normal), and Monte Carlo Simulation. Furthermore, the system includes a robust backtesting module evaluating model validity via the Kupiec Unconditional Coverage Test.

1.1 Design Philosophy

- **Decoupling:** Financial instruments, pricing engines, and risk calculation logic are separated into distinct classes to ensure maintainability.
- **Performance:** The system utilizes `NumPy` vectorized operations for high-performance matrix calculations and scenario generations. External heavyweight dependencies like `SciPy` are explicitly avoided (e.g., implementing Acklam's approximation for inverse cumulative distribution).
- **Extensibility:** The `Position` and Pricing structures are designed to easily accommodate new derivative types (e.g., American options) in future iterations.

2. Architecture and Core Modules

The software is structured around three main conceptual blocks: Utility Mathematics, Instrument Representation & Pricing, and the Risk Engine.

2.1 Mathematical Utilities (Utility Math Functions)

Provides lightweight, dependency-free statistical functions.

- `_norm_cdf & _norm_pdf` : Standard normal cumulative distribution and probability density functions utilizing Python's built-in `math.erf` .
- `_norm_ppf` : Implements Acklam's inverse-normal approximation. This is a deliberate design choice to achieve computational efficiency during Monte Carlo and Parametric parameter generation without relying on the `scipy.stats` library.

2.2 Instrument Representation & Pricing (Instruments)

- **`Position` (Dataclass):** A standardized schema representing a single portfolio row. It dynamically handles both linear (Stock) and non-linear (Option) assets. It defines the mapping between an instrument and its underlying risk factor via the `risk_symbol` property.
- **`BlackScholesPricer` :** A static utility class implementing the European Black-Scholes-Merton pricing model.

- `price()` : Computes the theoretical value of Calls and Puts. It includes boundary condition checks for expired options (maturity <= 0) or zero volatility scenarios.
- `delta()` : Computes the first-order derivative (Greek Delta) used extensively in the Parametric VaR methodology.

2.3 Core Risk Engine (`RiskCalculationSystem`)

The central class orchestrating data ingestion, calibration, and risk metric computation.

2.3.1 Data Ingestion and Handling

- **Input Data:** Accepts arbitrary `pandas.DataFrame` formats for portfolios and daily price histories, enforcing schema validation via `_coerce_portfolio` and `_coerce_price_history` .
- **Risk Factors:** The system identifies unique underlying assets (`risk_symbols`) from the portfolio and maps all historical returns and covariance matrices exclusively to these factors, ensuring dimensionality reduction.

2.3.2 Calibration (`calibrate_from_history`)

Transforms raw price history into statistical parameters needed for Parametric and Monte Carlo models:

- Calculates daily mean returns (`mu_daily`).
- Calculates the covariance matrix (`cov_daily`).
- Computes annualized historical volatility (`vol_annual`) which serves as a fallback if implied volatility is not explicitly provided in the portfolio input.

3. Algorithmic Design and Modeling Choices

3.1 Historical VaR and ES

- **Methodology:** Full Revaluation approach.
- **Mechanism:** Generates a set of future scenarios (`scenario_spots`) by applying either relative (log-returns) or absolute historical shocks to current spot prices. The entire portfolio is re-priced under each scenario using the `BlackScholesPricer` .
- **Risk Metrics:** VaR is extracted using linear interpolation at the specified confidence quantile. ES is computed as the conditional average of all tail losses exceeding the VaR threshold.

3.2 Parametric VaR and ES

- **Methodology:** Delta-Normal Approximation.
- **Mechanism:** To achieve closed-form analytical solutions, the system linearizes option positions using their Black-Scholes Delta (`_delta_exposures()`).
- **Calculation:** Computes portfolio-level mean and variance using the delta vector and the historical Covariance Matrix. VaR and ES are then derived directly from the properties of the normal distribution.

3.3 Monte Carlo VaR and ES

- **Methodology:** Multivariate Normal Simulation with Full Revaluation.
- **Mechanism:** Uses `numpy.random.default_rng` to draw random shock vectors from a multivariate normal distribution defined by the calibrated mean vector and covariance matrix.
- **Pricing:** Non-linear dependencies are accurately captured as the system performs a full Black-Scholes revaluation for all generated spot price paths.

3.4 Backtesting Module

- **Methodology:** Rolling-Window Backtesting.
- **Mechanism:** Iterates over the historical dataset, defining a rolling calibration window (e.g., 252 days). It calculates the 1-day VaR on day t and compares it against the realized PnL on day $t + 1$.
- **Validation:** Implements the Kupiec Unconditional Coverage Test (`_kupiec_test`), computing the Likelihood Ratio (LR) statistic and p-value to statistically accept or reject the model's accuracy at the specified tail probability.

4. Interfaces and Data Flow

1. **Initialization:** User instantiates `RiskCalculationSystem` with a Portfolio CSV and a Historical Prices CSV.
2. **Processing:** System auto-calibrates the Covariance Matrix and return vectors.
3. **Execution:** User invokes one of the VaR methods (`historical_var_es` , etc.), passing parameters like `confidence` or `n_sims` .
4. **Output:** Methods return a structured dictionary containing `current_value` , `var` , `es` , and internal statistics (e.g., `num_scenarios`), which can be directly parsed for reporting.

5. Known Limitations

- **Parametric Non-Linearity:** The Parametric method uses Delta-Normal mapping, ignoring second-order effects (Gamma). This will underestimate tail risk for portfolios with significant optionality convexity.
- **Distributional Assumptions:** The Monte Carlo and Parametric engines assume normal distributions for log-returns, which fails to capture empirical market "fat tails" (kurtosis). This choice limits the accuracy during extreme stress scenarios.

In [ ]:
```
1 d
```